

Desafio Prático - Aula 07

Esta atividade prática propõe o desenvolvimento do *firmware* para um nó sensor IoT em linguagem C embarcada, aplicando diretamente os conceitos de modularização, escopo e manipulação segura de memória vistos na Aula 07.

Em vez de focar apenas em sintaxe, o exercício simula um cenário real de engenharia onde você deve tomar decisões de projeto: projetar estruturas otimizadas para economizar memória (minimizando o *padding*), organizar o código em arquivos de interface (.h) e implementação (.c), proteger dados usando qualificadores adequados (*static*, *const*) e garantir a integridade do sistema ao banir a alocação dinâmica (*malloc*). Ao final, a atividade desenvolve a capacidade de análise crítica através da correção de um erro clássico em sistemas embarcados: o ponteiro pendente (*dangling pointer*).

Sua missão

Uma startup de IoT precisa do firmware de um novo nó sensor. Você é quem vai decidir qual sensor construir, desenhar a estrutura de dados, escrever a interface pública e implementar a lógica — aplicando tudo que viu na Aula 07: modularização, *static*/*volatile*/*const*, ponteiros seguros e o banimento do *malloc*.

Diferente de um roteiro de "copie e cole": aqui você vai tomar decisões de projeto, justificá-las, e no final até caçar um bug de verdade.

Plataforma: <https://onlinegdb.com> - selecione a linguagem C no menu Language.

Passo 0 — Escolha o sensor do seu nó IoT

Antes de programar, todo engenheiro toma decisões de projeto. Escolha UM dos sensores abaixo (ou proponha outro, combinando com o professor) — ele será a base de todo o seu módulo:

Sensor	Faixa do ADC (0–4095)	Fórmula de conversão sugerida
Temperatura (°C)	0 a 4095	$((adc * 3.3) / 4095) * 100$
Umidade relativa (%)	0 a 4095	$(adc / 4095.0) * 100$
Luminosidade (lux, aprox.)	0 a 4095	$(adc / 4095.0) * 1000$
Nível de um tanque (%)	0 a 4095	$(adc / 4095.0) * 100$

Anote sua escolha — você vai usá-la em todos os passos seguintes:

☐ Sensor escolhido: _____ Nome do módulo (ex.: umidade, luminosidade...): _____

Passo 1 — Configurar o ambiente

1. Acesse onlinegdb.com.

2. No menu Language (canto superior direito), selecione C.

3. No painel esquerdo (Gerenciador de Arquivos), clique em New File e crie dois arquivos com o nome do seu módulo, por exemplo: umidade.h e umidade.c (troque "umidade" pelo nome do sensor que você escolheu no Passo 0).

Passo 2 — Projete a sua estrutura de dados (sem código pronto!)

Antes de escrever qualquer linha, decida: quais informações o seu sensor precisa entregar? Pense em pelo menos estes quatro campos:

- Um identificador do sensor (existe mais de um sensor desse tipo na estação?)

- O valor medido, já convertido (ex.: temperatura, umidade...)
- Um instante de tempo (timestamp) da leitura
- Um indicador de "leitura válida ou não"

Aplique o que você aprendeu sobre padding

Na aula você viu que a ordem dos campos de uma struct afeta o tamanho final dela na memória. Antes de digitar sua struct, faça no papel (ou em comentário no próprio código) a conta de quantos bytes cada campo ocupa e escreva os campos do maior para o menor, minimizando o padding.

Agora, no arquivo <nome>.h, escreva sua própria typedef struct com os campos que você decidiu, na ordem que você calculou. Não existe uma única resposta certa — existe a estrutura que você projetou e consegue justificar.

Passo 3 — Construa a interface pública (.h)

Complete o arquivo <nome>.h seguindo este checklist. Você escreve o código; aqui vão apenas as regras que ele precisa cumprir:

- ☐ Include guards com #ifndef / #define / #endif, usando um nome único baseado no seu módulo (ex.: UMIDADE_H).
- ☐ #include <stdint.h> e #include <stdbool.h>.
- ☐ A struct que você projetou no Passo 2, com um nome do tipo <nome>_t.
- ☐ Protótipo de uma função de inicialização: void <nome>_init(void);
- ☐ Protótipo de uma função de leitura segura, que recebe um ponteiro para a struct (Caller-Allocated Buffer) e devolve bool indicando sucesso — pense no nome e nos parâmetros que fazem sentido para o seu sensor.
- ☐ Protótipo de uma função que devolve o total de leituras já realizadas (uint32_t).

Passo 4 — Implemente a lógica privada (.c)

No arquivo <nome>.c, implemente as três funções que você declarou no .h. Antes de codificar, responda estas perguntas de projeto (pode ser em comentário no topo do arquivo):

1. Qual variável deve guardar o total de leituras? Ela deve ser static ou não? Por quê?
 2. A função que converte o valor bruto do ADC para a unidade do seu sensor precisa ser visível fora do arquivo .c? Ela deve levar static?
 3. Onde deve ficar a checagem de ponteiro nulo, e o que sua função deve fazer se ela falhar?
- Use a fórmula de conversão que você escolheu no Passo 0 (ou crie a sua própria, se preferir simular outro tipo de sensor).

Dica, não solução

A função de leitura deve seguir o padrão do Caller-Allocated Buffer: quem chama já criou a struct; sua função só valida o ponteiro e preenche os campos. Reveja o exemplo de ler_pressao_seguro() na Aula 07 se precisar relembrar o formato — mas adapte os nomes e os campos para o seu sensor.

Passo 5 — main.c: coloque seu módulo para funcionar

Escreva um main.c que:

- Inclua seu arquivo <nome>.h e chame a função de inicialização.
- Faça um laço simulando pelo menos 5 leituras do sensor (use uint8_t como no exemplo da aula).
- Guarde cada valor lido em um array (não em 5 variáveis separadas!).

- Depois do laço, use — ou adapte — a função `calcular_media()` vista na Aula 07 para calcular a média das leituras, passando o array e o seu tamanho como parâmetros.
- Imprima cada leitura individual e, ao final, a média calculada.

Desafios bônus — para ir além

Desafio A — Alarme com const

Defina um limite fixo (ex.: temperatura máxima segura) usando `const`. Depois de cada leitura, compare o valor lido com esse limite e imprima um aviso caso ele seja ultrapassado. Justifique em um comentário por que esse limite deve ser `const` e não apenas uma variável comum.

Desafio B — Histórico sem malloc

Implemente um pequeno pool de memória estática (como o exemplo `pool_memoria_t` da Aula 07) para guardar o histórico das últimas leituras do seu sensor, sem usar `malloc`. Descreva, em um comentário, o que aconteceria se você tivesse usado `malloc()` nesse cenário rodando 24 horas por dia em um dispositivo real.

Desafio C — Caça ao bug: o ponteiro pendente

O trecho abaixo foi escrito por um colega de equipe e não compila com segurança — ele tem exatamente o problema que a Aula 07 chamou de "ponteiro pendente". Encontre o erro, explique em uma frase por que ele é perigoso, e reescreva a função seguindo o padrão correto (Caller-Allocated Buffer):

```
// Código do colega -- NÃO está seguro. Onde está o problema?  
uint32_t* ler_sensor_luminosidade(void) {  
    uint32_t leitura = 850; // valor simulado do ADC  
    return &leitura;  
}
```

Teste e validação

1. Clique no botão verde Run no OnlineGDB.
2. Confira no console: as leituras individuais aparecem? A média foi calculada corretamente?
3. Se você fez o Desafio A: o alarme dispara quando o valor ultrapassa o limite?
4. Se você fez o Desafio C: sua versão corrigida roda sem comportamento estranho (valores "lixo")?

Checklist final de revisão (code review)

- ☐ Include guards presentes e com nome único no arquivo `.h`.
- ☐ Tipos exatos: `uint8_t`, `uint32_t`, etc. em vez de `int` genérico.
- ☐ A struct foi organizada do maior para o menor campo (padding minimizado) - e você sabe explicar por quê.
- ☐ A função de conversão e a variável de contagem estão `static` no `.c`, e você sabe justificar essa escolha.
- ☐ A função de leitura valida o ponteiro (`== NULL`) antes de escrever nele.
- ☐ O array de leituras foi passado para `calcular_media()` junto com o seu tamanho (sem depender de array decay).
- ☐ Nenhuma chamada a `malloc` ou `free` em qualquer lugar do projeto.
- ☐ Você consegue explicar, em uma frase, cada decisão de projeto que tomou (struct, static, const).

Para entregar: envie os códigos `telemetria.h`, `telemetria_<sensor>.c` e `main.c`, junto com as respostas escritas dos Passos 2 e 4 e do Desafio C no **AVA**.